



Lab 13

File Handling Basics

<https://github.com/mmujtaba25/cs-212>

Muhammad Mujtaba

CMD ID: 540040

mmujtaba.bese25seecs@seecs.edu.pk

Class: BESE 16B

Batch: 2k25

Lab 13 - Main

```
package Lab13;

import java.io.*;
import java.util.*;

public class Main
{
    private static final Scanner IN = new Scanner(System.in);
    private static final String FILE_NAME = "marks.txt";
    private static final int BOX_WIDTH = 50;

    public static void main(String[] args)
    {
        // --no-modern for old consoles
        if (Arrays.stream(args).anyMatch(arg → arg.equalsIgnoreCase("--no-modern")
|| arg.equalsIgnoreCase("-nm")))
            ConsoleUtils.setModernMode(false);

        try (StudentRecords records = new StudentRecords(FILE_NAME))
        {
            printMenu();

            // if records empty, add 5 sample records
            if (records.getAllRecords().isEmpty())
            {
                records.appendRecord("Sample Alice", 85.5);
                records.appendRecord("Sample Bob", 92.0);
                records.appendRecord("Sample Charlie", 78.0);
                records.appendRecord("Sample Diana", 88.5);
                records.appendRecord("Sample Ethan", 91.0);
            }

            while (true) if (!runLoopIteration(records)) break;
        }
        catch (IOException e)
        {
            System.err.println("File error: " + e.getMessage());
        }
    }

    private static boolean runLoopIteration(StudentRecords records) throws
IOException
    {
        MenuOption option = getMenuOption();
        ConsoleUtils.oneLineSpace();

        switch (option)
        {
            case ADD_RECORD → addRecord(records);
            case ADD_MULTIPLE → addMultipleRecords(records);
            case SHOW_ALL → showAllRecords(records);
            case SEARCH → searchRecord(records);
        }
    }
}
```

```

        case STATS → printStats(records);
        case HELP → printMenu();
        case EXIT →
        {
            ConsoleUtils.printBox("Goodbye!", BOX_WIDTH);
            return false;
        }
        default → System.out.printf(
            "> Invalid input. Please enter a number between %d and %d.%n",
            MenuOption.firstIndex(),
            MenuOption.lastIndex()
        );
    }

    ConsoleUtils.oneLineSpace();
    return true;
}

/* MENU */

private static void printMenu()
{
    final String[] options = MenuOption.getAllString();
    ConsoleUtils.printBox("Student Records Menu", options, BOX_WIDTH);
    ConsoleUtils.oneLineSpace();
}

/* OPERATIONS */

private static void addRecord(StudentRecords records) throws IOException
{
    String name = getInputFromUser("Enter student name");
    if (name.isEmpty())
    {
        System.out.println("Name cannot be empty.");
        return;
    }

    String marksStr = getInputFromUser("Enter marks");
    try
    {
        double marks = Double.parseDouble(marksStr);
        records.appendRecord(name, marks);
        ConsoleUtils.printBox("Success", new String[]{"+ Record added: " + name
+ " - " + marks}, BOX_WIDTH);
    }
    catch (NumberFormatException e)
    {
        System.out.println("Invalid marks format. Please enter a valid
number.");
    }
}

private static void addMultipleRecords(StudentRecords records) throws

```

```

IOException
{
    ConsoleUtils.printBox(
        "Add Multiple Records", new String[]{
            "+ Enter records in format: Name Marks", "+ Type 'done' to
finish"
        }, BOX_WIDTH
    );
    ConsoleUtils.oneLineSpace();

    int added = 0;
    while (true)
    {
        System.out.print("Record: ");
        String input = IN.nextLine().trim();

        if (input.equalsIgnoreCase("done")) break;
        if (input.isEmpty()) continue;

        String[] parts = input.split("\\s+");
        if (parts.length ≥ 2)
        {
            try
            {
                String name = parts[0];
                double marks = Double.parseDouble(parts[1]);
                records.appendRecord(name, marks);
                System.out.println("  Added: " + name + " - " + marks);
                added++;
            }
            catch (NumberFormatException e)
            {
                System.out.println("  Invalid marks. Use format: Name Marks");
            }
        }
        else
        {
            System.out.println("  Invalid format. Use: Name Marks");
        }
    }
    System.out.println("\nAdded " + added + " record(s).");
}

private static void showAllRecords(StudentRecords records) throws IOException
{
    List<Student> students = records.getAllRecords();

    if (students.isEmpty())
    {
        ConsoleUtils.printBox("All Records", new String[]{"+ No records
found."}, BOX_WIDTH);
        return;
    }
}

```

```

        String[] lines = new String[students.size()];
        for (int i = 0; i < students.size(); i++)
        {
            lines[i] = "+ " + students.get(i).stringify();
        }

        ConsoleUtils.printBox("All Records (" + students.size() + ")", lines,
BOX_WIDTH);
    }

    private static void searchRecord(StudentRecords records) throws IOException
    {
        String name = getInputFromUser("Enter student name to search");

        if (name.isEmpty())
        {
            System.out.println("Search name cannot be empty.");
            return;
        }

        List<Student> results = records.searchByName(name);

        if (results.isEmpty())
        {
            ConsoleUtils.printBox("Search Results", new String[]{"+ No records found
for '" + name + "'"}, BOX_WIDTH);
        }
        else
        {
            String[] lines = new String[results.size()];
            for (int i = 0; i < results.size(); i++)
            {
                lines[i] = "+ " + results.get(i).stringify();
            }
            ConsoleUtils.printBox("Search Results for '" + name + "'", lines,
BOX_WIDTH);
        }
    }

    private static void printStats(StudentRecords records) throws IOException
    {
        int count = records.countRecords();

        if (count == 0)
        {
            ConsoleUtils.printBox("Statistics", new String[]{"+ No records for
statistical analysis."}, BOX_WIDTH);
            return;
        }

        List<String> lines = new ArrayList<>();
        lines.add("+ Total Records: " + count);

        OptionalDouble avg = records.averageMarks();

```

```

        if (avg.isPresent()) lines.add(String.format("+ Average Marks: %.2f",
avg.getAsDouble()));

        records.highestMarks().ifPresent(student → lines.add("+ Highest: " +
student.stringify()));
        records.lowestMarks().ifPresent(student → lines.add("+ Lowest: " +
student.stringify()));

        ConsoleUtils.printBox("Statistics", lines.toArray(new String[0]),
BOX_WIDTH);
    }

    /* HELPERS */

    private static String getInputFromUser(String prompt)
    {
        System.out.print(prompt + ": ");
        return IN.nextLine();
    }

    private static MenuOption getMenuOption()
    {
        final int option;
        System.out.printf(
            "Select an option between %d and %d (%d for help): ",
            MenuOption.firstIndex(),
            MenuOption.lastIndex(),
            MenuOption.HELP.getIndex()
        );
        try { option = Integer.parseInt(IN.nextLine()); }
        catch (Exception e) { return MenuOption.INVALID; }
        return MenuOption.fromIndex(option);
    }

    /* MENU OPTION ENUM */

    private enum MenuOption
    {
        HELP(0),
        ADD_RECORD(1),
        ADD_MULTIPLE(2),
        SHOW_ALL(3),
        SEARCH(4),
        STATS(5),
        EXIT(6),
        INVALID(-1);

        private final int index;

        MenuOption(int index) { this.index = index; }

        public static MenuOption fromIndex(int index)
        {
            for (MenuOption option : values())

```

```

        if (option.index == index) return option;
        return MenuOption.INVALID;
    }

    public int getIndex() { return index; }

    public static int firstIndex() { return HELP.getIndex(); }

    public static int lastIndex() { return EXIT.getIndex(); }

    public static MenuOption[] getAll()
    {
        return Arrays.stream(values()).filter(option → option !=
INVALID).toArray(MenuOption[]::new);
    }

    public static String[] getAllString()
    {
        return
Arrays.stream(getAll()).map(MenuOption::getAllFormat).toArray(String[]::new);
    }

    private static String getAllFormat(MenuOption option)
    {
        return option.getIndex() + ". " + option.name().charAt(0) +
option.name()
                .substring(1)
                .toLowerCase()
                .replace("_", " ");
    }
}

```

Program Output

Student Records Menu
0. Help 1. Add record 2. Add multiple 3. Show all 4. Search 5. Stats 6. Exit

Select an option between 0 and 6 (0 for help): 3

All Records (5)
+ SampleAlice - 85.5 + SampleBob - 92.0

+ SampleCharlie - 78.0
+ SampleDiana - 88.5
+ SampleEthan - 91.0

Select an option between 0 and 6 (0 for help): 1

Enter student name: ConsoleAAlice^1

Enter marks: 67nsole Alice ^[

Success

+ Record added: Console Alice 1 - 67.0
--

Select an option between 0 and 6 (0 for help): 2

Add Multiple Records

+ Enter records in format: Name Marks
+ Type 'done' to finish

Record: ConsoleBob 34

Added: ConsoleBob - 34.0

Record: ConsoleCharlie 876

Added: ConsoleCharlie - 87.0

Record: done

Added 2 record(s).

Select an option between 0 and 6 (0 for help): 0

Student Records Menu

0. Help
1. Add record
2. Add multiple
3. Show all
4. Search
5. Stats
6. Exit

Select an option between 0 and 6 (0 for help): 3

Skipping invalid line: SampleEthan - 91.0Console Alice 1 - 67.0

All Records (6)

+ SampleAlice - 85.5

```
+ SampleBob - 92.0
+ SampleCharlie - 78.0
+ SampleDiana - 88.5
+ ConsoleBob - 34.0
+ ConsoleCharlie - 87.0
```

Select an option between 0 and 6 (0 for help): 4

Enter student name to search: bob

Skipping invalid line: SampleEthan - 91.0Console Alice 1 - 67.0

Search Results for 'bob'
+ SampleBob - 92.0
+ ConsoleBob - 34.0

Select an option between 0 and 6 (0 for help): 4[[A

Enter student name to search: bob

Search Results for 'bob'
+ SampleBob - 92.0
+ ConsoleBob - 34.0

Select an option between 0 and 6 (0 for help): 0

Student Records Menu
0. Help
1. Add record
2. Add multiple
3. Show all
4. Search
5. Stats
6. Exit

Select an option between 0 and 6 (0 for help): 5

Statistics
+ Total Records: 8
+ Average Marks: 77.88
+ Highest: SampleBob - 92.0
+ Lowest: ConsoleBob - 34.0

Select an option between 0 and 6 (0 for help): 6

Goodbye!

Lab 13 - Student

```
package Lab13;

public record Student(String name, double marks)
{
    public static final String SEPARATOR = "-";

    public Student(String name, double marks)
    {
        // only numbers and alphabets with spaces
        this.name = name.replaceAll("[^a-zA-Z0-9 ]", "");
        this.marks = marks;
    }

    public static Student from(String name, double marks) { return new Student(name, marks); }

    public static Student from(String name, String marksStr) throws
    NumberFormatException
    {
        double marks = Double.parseDouble(marksStr.trim());
        return new Student(name, marks);
    }

    public String stringify() { return name + " " + SEPARATOR + " " + marks; }
}
```

Lab 13 - StudentRecords

```
package Lab13;

import java.io.*;
import java.util.*;

public class StudentRecords implements AutoCloseable
{
    private final String fileName;
    private final BufferedWriter writer;

    public StudentRecords(String fileName) throws IOException
    {
        this.fileName = fileName;
        this.writer = new BufferedWriter(new FileWriter(fileName, true));
    }

    @Override
    public void close() throws IOException
    {
        writer.close();
    }

    public void appendRecord(String name, double marks) throws IOException
    {
        Student student = Student.from(name, marks);
        writer.write(student.stringify());
        writer.newLine();
        writer.flush();
    }

    public List<Student> getAllRecords() throws IOException
    {
        List<Student> students = new ArrayList<>();

        try (BufferedReader reader = new BufferedReader(new FileReader(fileName)))
        {
            String line;
            while ((line = reader.readLine()) != null)
            {
                line = line.trim();
                if (line.isEmpty()) continue;

                String[] parts = line.split(Student.SEPARATOR);
                if (parts.length >= 2)
                {
                    try { students.add(Student.from(parts[0], parts[1])); }
                    catch (NumberFormatException e)
                    {
                        System.out.println("Skipping invalid line: " + line);
                    }
                }
            }
        }
    }
}
```

```

    }
    return students;
}

public List<Student> searchByName(String name) throws IOException
{
    List<Student> students = getAllRecords();
    List<Student> results = new ArrayList<>();
    for (Student s : students)
        if (s.name().toLowerCase().contains(name.toLowerCase())) results.add(s);
    return results;
}

public int countRecords() throws IOException
{
    return getAllRecords().size();
}

public OptionalDouble averageMarks() throws IOException
{
    return getAllRecords().stream().mapToDouble(Student::marks).average();
}

public Optional<Student> highestMarks() throws IOException
{
    return
    getAllRecords().stream().max(Comparator.comparingDouble(Student::marks));
}

public Optional<Student> lowestMarks() throws IOException
{
    return
    getAllRecords().stream().min(Comparator.comparingDouble(Student::marks));
}
}

```

Lab 13 - ConsoleUtils

```
package Lab13;

public class ConsoleUtils
{
    private static boolean modernMode = true;

    public static void setModernMode(boolean modernMode) { ConsoleUtils.modernMode = modernMode; }

    /* SYMBOLS */

    private static String h() { return modernMode ? "─" : "-"; }

    private static String v() { return modernMode ? "│" : "|"; }

    private static String tl() { return modernMode ? "└" : "+"; }

    private static String tr() { return modernMode ? "┐" : "+"; }

    private static String bl() { return modernMode ? "└" : "+"; }

    private static String br() { return modernMode ? "┐" : "+"; }

    private static String ml() { return modernMode ? "├" : "+"; }

    private static String mr() { return modernMode ? "┤" : "+"; }

    /* BOX */

    public static void printTopBorder(int width) { println(tl() + repeat(h(), width) + tr()); }

    public static void printMiddleBorder(int width) { println(ml() + repeat(h(), width) + mr()); }

    public static void printBottomBorder(int width) { println(bl() + repeat(h(), width) + br()); }

    public static void printCenteredLine(String text, int width) { println(v() + centerText(text, width) + v()); }

    public static void printLeftLine(String text, int width)
    {
        println(v() + " " + padRight(text, width - 2) + " " + v());
    }

    /* COMPONENTS */

    public static void printBox(String title, String[] lines, int width)
    {
        printTopBorder(width);
```

```

        if (title != null && !title.isEmpty())
        {
            printCenteredLine(title, width);
            printMiddleBorder(width);
        }

        for (String line : lines)
        {
            printLeftLine(line, width);
        }

        printBottomBorder(width);
    }

    public static void printBox(String title, int width)
    {
        printTopBorder(width);

        if (title != null && !title.isEmpty())
        {
            printCenteredLine(title, width);
        }

        printBottomBorder(width);
    }

    /* GENERAL */

    public static String repeat(String s, int count) { return s.repeat(Math.max(0,
count)); }

    public static String centerText(String text, int width)
    {
        if (text.length() ≥ width) return text;

        int padding = width - text.length();
        int left = padding / 2;
        int right = padding - left;

        return " ".repeat(left) + text + " ".repeat(right);
    }

    public static String padRight(String text, int width)
    {
        if (text.length() ≥ width) return text;
        return text + " ".repeat(width - text.length());
    }

    /* INTERNAL */

    private static void println(String s) { System.out.println(s); }

    public static void oneLineSpace() { System.out.println(); }
}

```